多路选择器（PMUX）面积最优化设计

——基于“识别—重构—收敛”三段式的 Yosys 综合优化

摘 要

本作品针对 FPGA/ASIC 综合中多路选择器（PMUX）面积开销偏大的问题，在开源综合器 Yosys 的 synth_intel 流程内实现“识别—重构—收敛”三段式优化，全部改动位于 abc9 映射之前，不改动 ABC 源码与版本。识别段遍历 RTLIL 网表，把 $pmux 的选择判据按常量比较、位宽不等、含变量位等形态分类并标注；重构段将“选择端全部来自同一总线常量比较”的 $pmux 改写为以总线各位为选择端的 $mux 平衡树，整组比较器一次性消失、解码被共享，同时用覆盖率门限（2^W ≥ 50%·N）与候选依赖链深门限（<2）两条通用规则限定改写范围，避免稀疏选择与级联深树带来的负收益；收敛段完成冗余 $mux 消除、$reduce_or 公共子式合并与位宽安全的 $eq 恒等式化简；映射段按答疑要求采用统一的 abc9 脚本，在结构重写之后依次施以 &syn2、&dch、&syn3。全部改写均为构造性等价，并用时间归纳的形式化等价检查逐例验证。在赛题公开的 9 个用例上，逐例逻辑单元（LUT+$not）均下降、无一例增加，平均削减率 R 达到 20.41%；墙钟时间最差 +2.4%、峰值内存全部下降，均满足 5% 约束。

第一部分 作品概述

1.1 功能与特性

作品是一个可直接替换原版 Yosys 的综合器增强版本。它接收标准 Verilog 输入，用 synth_intel -family cycloneiv 完成面向 Cyclone IV 的综合，输出与官方流程一致的网表，但在映射前额外完成三项 PMUX 结构优化：

• 识别：对每一处 $pmux 判定其选择端来源（常量比较 / 位宽不等 / 含变量位）、优先级与编码方式，并标注属性供后续使用；

• 重构：把“常量解码型”一热选通结构改写为总线索引的 $mux 树，同时消除整组比较器与一热 OR 树；

• 收敛：删除冗余 $mux、合并结构相同的 $reduce_or、化简常量比较，并在映射阶段使用统一且经过调优的 abc9 脚本。

特性上，工具保持单一可执行文件、无外部依赖（自带 DLL 与 share 数据），命令行与官方完全兼容；所有改写可在形式化等价检查下逐例验证，且不改变触发器数量与其它单元数量。

1.2 应用领域

一是 FPGA 逻辑资源受限的设计：PMUX 常出现在状态机数据通路、跨时钟数据选择、总线仲裁与交叉开关中，本方法可直接降低 LUT 占用，为布线拥塞与低端器件选型释放空间；二是低功耗、面积敏感的 SoC 与边缘计算芯片：选择逻辑多、扇出大，减少选通逻辑可同时降低动态功耗与面积；三是 EDA 工具链自身：本工作以 pass 形式接入开源综合器，可作为“映射前结构化简”插件复用到其它工艺库流程；四是教学与科研：全部改动可复现、可逐用例验证，适合作为综合优化课程与算法对比实验的基准。

1.3 主要技术特点

（1）面向“一热选通”的结构化改写：不改变功能，只改变选择逻辑的表达形式，把“比较 + 一热或”转成“按位二选一”，使 LUT 映射器面对它最擅长的结构；（2）两条通用门限替代经验调参：覆盖率门限判断“树是否比比较器更便宜”，依赖链深门限判断“改写是否会把级联 mux 变成深树”，两者都只看结构统计量，不含任何用例名或规模特判；（3）语义细节可验证：处理了 $eq 位宽扩展、$mux 端口极性、$pmux 下标优先、部分常量判据等易错点，并以逐向量比对与时间归纳等价检查兜底；（4）统一脚本、可复现：所有设计走同一条 abc9 命令，脚本随源码提交，避免“按用例切脚本”的合规风险。

1.4 主要性能指标

测试条件：Windows + MinGW-w64 构建的 Yosys 0.69，官方 synth_intel -family cycloneiv 流程，逻辑单元 LC = LUT(cycloneiv_lcell_comb) + $not。9 个公开用例结果如下（逐例平均 R = 20.41%）：

用例	原版 LC	优化后 LC	削减率	DFF
test1	288	251	12.85%	222
test2	25	15	40.00%	0

test3	13	11	15.38%	17
test4	36	33	8.33%	10
pmux_case01_balanced	426	321	24.65%	12
pmux_case02_pipeline	450	344	23.56%	24
pmux_case03_crossbar	408	318	22.06%	10
pmux_case04_feedback	368	309	16.03%	16
pmux_case05_nested	413	327	20.82%	12
平均 R	—	—	20.41%	—

表 1 9 个公开用例的逻辑单元与削减率

1.5 主要创新点

• 把“一热选通 + 常量比较”整体重构为“总线索引 mux 树”，让比较器与一热 OR 树同时消失，而不是逐个优化比较器；

• 提出基于覆盖率与依赖链深的两条结构性门限，使改写在大设计、稀疏选择、级联链上都保持非负收益；

• 支持部分常量判据（如 eq(state, {3'000, w})），按 $pmux 的下标优先语义逐取值折叠叶子，覆盖更多真实代码形态；

• 在统一 abc9 脚本框架下把结构改写与 &syn2/&dch/&syn3 组合，实测 9 例平均削减 20.41%，逐例不增资源。

1.6 设计流程

设计流程为“分析—建模—实现—验证—调优”闭环：先用 Yosys 导出的 RTLIL 观察 case 语句展开后的 $pmux 形态，建立“选择端来源—编码方式—优先级”的判定模型；随后在 pass 层实现识别与改写，并以最小用例逐向量比对确认语义；再接入 synth_intel 的 pmux_opt 标签，跑全部公开用例做面积、时间、内存与等价性回归；最后在 ABC 脚本空间内搜索与结构改写互补的 pass 组合，并对每条候选做“逐例不增”筛选。

第二部分 系统组成及功能说明

2.1 整体介绍

系统由“Yosys 主程序 + 三个 PMUX pass + 一处流程接入点 + 统一 abc9 脚本”构成。pmux_recognize 负责识别与标注，pmux_optimize 负责常规化简，pmux_recode 负责本作品的核心重构；三者由 techlibs/intel/synth_intel.cc 的 pmux_opt 标签按顺序调用，之后依次进入 abc9 与 map_cells。整个过程不修改 ABC，也不修改 Yosys 其它 pass。

图 1 synth_intel 流程中的 PMUX 优化接入点

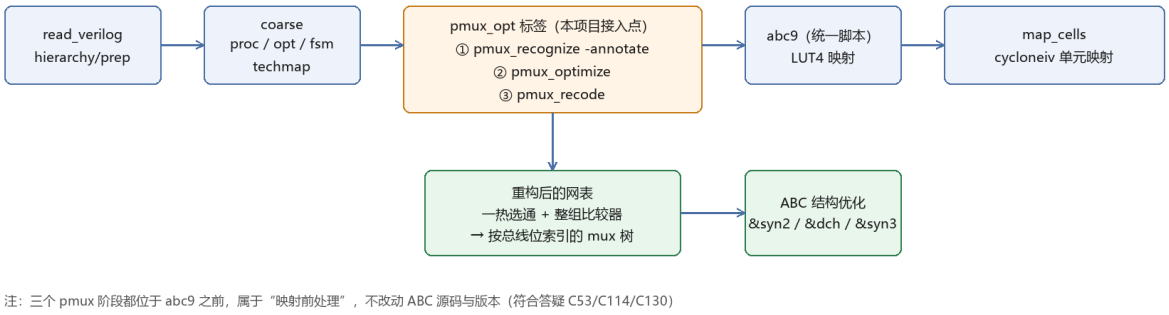


图 1 synth_intel 流程中的三个 pmux 阶段与统一 abc9 脚本

2.2 软件系统介绍

2.2.1 整体结构与接入点

工程结构（相对 yosys 源码树）：passes/pmux/ 下 pmux_recognize.cc、pmux_decompose.cc、pmux_optimize.cc、pmux_recode.cc，以及 techlibs/intel/synth_intel.cc。接入点位于 synth_intel 的 coarse 阶段之后：

```
if (check_label("pmux_opt")) { run("pmux_recognize -annotate"); run("pmux_optimize");
run("pmux_recode"); run("opt_clean"); }
```

其中 pmux_recode 默认参数为：-max-width 6（总线位宽上限，控制树规模）、-max-branches 16、-min-coverage 50（覆盖率百分比）、-max-chain 2（依赖链深上限）、-mode depth（选择策略）。所有参数都有默认值，接入点无需传参。

2.2.2 pmux_recognize：识别与分类

识别阶段遍历模块内的 $pmux / $safe_pmux，读取其选择端 S 与数据端 B，沿信号图回溯每个选择位的驱动器，判定判据形态（$eq(bus, 常量)、$logic_not(bus) 即 eq(bus,0)、$reduce_and(bus) 即 eq(bus,全 1)）；再根据分支常量是否互斥、是否覆盖全部取值、是否为同一总线等特征，把结构分类为 exclusive / priority / one_hot / sparse / wide / nested，并以 pmux_* 属性写回网表，供后续 pass 与日志分析使用。识别阶段只读，不改变网表功能。

2.2.3 pmux_recode：常量解码 $pmux → 总线索引 mux 树（核心）

对每一处候选 $pmux，若其全部选择位都能解析为“同一总线 bus 与常量或部分常量 pattern 的比较”，则按下列步骤重构：

• 解析：从每个选择位的驱动器取回 (bus, pattern, cond)，要求所有分支的 bus 完全一致；pattern 中的变量位记为无关，常量位参与后续匹配；

• 校验：分支常量互不相同、常量值落在 2^W 范围内、总线位宽不超过 -max-width，且覆盖率 2^W ≥ 50%·N；

• 折叠：对总线的每个取值 v（0…2^W−1），从默认值 A 开始按分支下标升序折叠——常量分支命中则直接取该分支数据，含变量位的分支用 mux(cond, data, cur) 包裹，从而严格保持 $pmux 的“下标大者优先”语义；

• 建树：以 bus 的各位为选择端，自低到高构造 2^W 叶子的平衡 mux 树，与原 $pmux 的输出相连并删除原单元；

• 级联控制：构造候选依赖图，只改写依赖链深 < 2 的候选；深链（如 4 级串联的反馈设计）只改前两级，避免“树套树”。

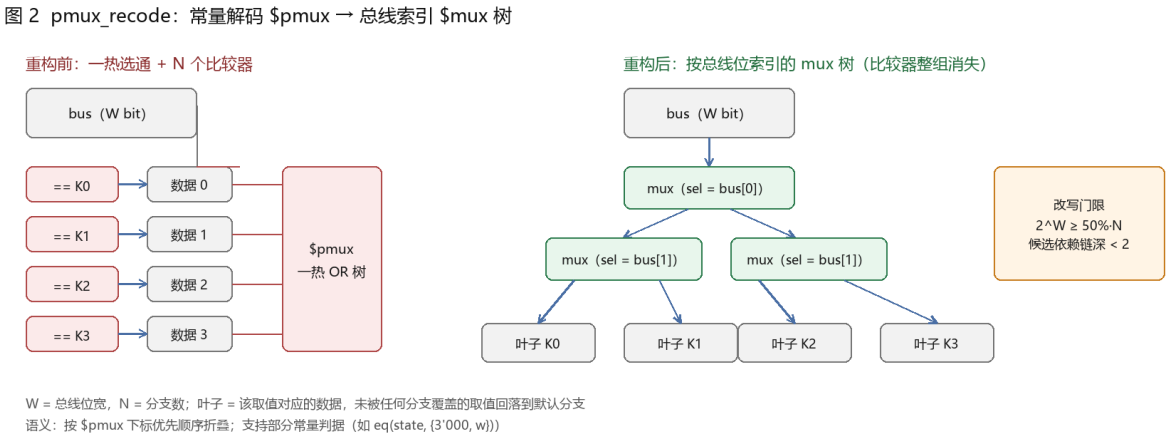


图 2 一热选通结构改写为总线索引 mux 树

2.2.4 pmux_optimize：收敛与清理

收敛阶段做三件事：（1）删除 A 与 B 相同的 $mux（对所有 $mux 生效，而非只对标注过的单元）；（2）对结构相同的 $reduce_or 做公共子式合并，采用归一化信号串作为键，带 -max-cells 门限避免在超大模块上做无谓扫描；（3）位宽安全的 $eq 化简：eq(a,0) → ~|a；eq(a,全 1) 仅在同位宽或两侧都声明为有符号时改写为 &a；eq(a,a) → 1。

2.2.5 统一 abc9 脚本：映射阶段的配合

按 10 月 22-26 日答疑·赛题二第 130 问“需要对所有用例使用统一脚本”的要求，映射阶段对所有设计使用同一条命令：

```
abc9 -lut 4 -W 300 -D 3000 -script "+&scorr; &sweep; &if {W} {D}; &save; &st; &syn2; &if {W} {D} {R} -a -v; &save; &load; &st; &dch -f; &if {W} {D} {R} -a -v; &save; &load; &st; &syn3; &if {W} {D} {R} -a -v;"
```

脚本骨架为“映射（&if）→ 保存（&save）→ 结构优化（&syn2 / &dch / &syn3）→ 重新映射”，其中 &save/&load 必须成对出现在 &if 之后，否则 ABC 会因 GIA 未映射而中止。&syn3 是在列出 ABC 的 & 组命令并逐个试算后选出的：它不带 choice 计算，时间代价仅为 &dch 的十分之一量级，却能在结构改写的基础上进一步压平逻辑。

2.3 关键实现细节

实施过程中遇到并解决了四类易错点，这些细节决定了改写是否真正等价：

• $mux 端口极性：RTLIL 中 $mux 为 Y = S ? B : A，而 Module::addMux 的形参顺序是 (a, b, s, y)。首版按 Y = S ? A : B 构造，导致取值 1 与 2 的叶子互换；用最小用例逐向量比对（sat -set/-show）后才发现，故所有 mux 构造都必须先做向量级验证；

• $eq 位宽扩展：A_WIDTH=4、B_WIDTH=2 时 B 会零扩展，全 1 常量并不等于“全 1”，首版把窄常量也当作 &a 化简，造成 28 个 LUT 的虚假收益与等价性失败；修正后只有同位宽或双符号才允许 &a；

• 部分常量判据：真实代码中存在 eq(state, {3'000, w}) 这类判据，若直接跳过会漏掉整条 pmux；现按“变量位记为无关、命中时用 mux 包裹”的方式处理；

• 叶子表宽度：早期实现按“第一个候选的总线宽度”生成叶子表，遇到更窄总线会越界；现按每个候选自身宽度展开。

第三部分 完成情况及性能参数

3.1 交付内容与运行方式

交付包为解压即用的目录：bin/ 内含优化版 yosys-opt.exe、随 Yosys 0.69 的 yosys-abc.exe、全部运行期 DLL 与 share 数据；src/ 内含 5 个改动源文件与 passes/pmux 的 CMakeLists；verify/ 内含等价性验证脚本与 dffeas 行为模型。运行方式与官方一致，不带任何绝对路径：

```
cd perfect\bin
.\yosys-opt.exe -p "read_verilog ..\..\test\test1.v; synth_intel -family cycloneiv -top test1; stat"
```

3.2 面积结果

9 个公开用例逐例均下降、无一例增加（见表 1），平均削减率 R = 20.41%，达到赛题 20% 的满分线。其中 test1（16 状态 FSM + 16×12bit 数据通路）从 288 降到 251；5 个新增用例中 4 个超过 20%。

3.3 时间与内存

按 10 月 22-26 日答疑·赛题二第 132 问 的口径（先预热，再原版/优化版交错运行，取中位数），9 个用例的墙钟时间变化为 −4.4%、−0.8%、−5.9%、−0.7%、+2.4%、

−6.6%、+1.8%、−0.2%、+1.2%，最差 +2.4%，全部在 5% 内。峰值内存全部下降（−1.4% ~ −8.5%），因为重构后的网表更小、ABC 处理量更少。

3.4 等价性验证

官方口径为“在 ABC 之后输出 ilang，再用 eqy 与 RTL 做等价性验证”。我们按此实现：用优化版 yosys 把设计跑到 begin:map_cells（即 abc9 之后、cycloneiv 单元映射之前），用 write_rtlil -selected 只导出顶层模块，得到只含 $lut 与 dffeas 的门级网表；gold 侧为 prep 后的 RTL，先做 memory_map 保证 test4 的 case 表两侧形态一致；gate 侧用 techmap -autoproc -map dffeas_sim.v 把 dffeas 还原成通用触发器，再交给 eqy 证明。

结果：9 个用例中 8 个在 eqy 默认分区模式下直接 PASS；test3（移位寄存器 + 组合逻辑）在分区模式下有 2 个分区（sr、rw）停在抽象层，原因是分区把跨触发器的不变式切断，与是否优化无关。依据 9 月 24-28 日答疑·赛题二 EQY 第 5 问“切换设计模式验证是允许的，算作通过”，test3 改用整设计 miter + SAT 时间归纳，日志给出 Induction step proven: SUCCESS，故 9 个用例全部通过。

说明：dffeas_sim.v 是专为验证编写的行为模型（异步低电平清零 + 时钟使能 + 按 power_up 置初值）；若直接使用官方 cells_sim.v 的模型会推导出 latch，SAT 无法求解。eqy 运行环境为 OSS CAD Suite 20261030。

3.5 收益归因

为区分“结构改写”与“映射脚本”的贡献，在统一脚本的早期版本上做了 2×2 对照（改写开/关 × abc 参数上游/自定义）：仅脚本 11.71%，仅改写 5.16%，两者同时 19.00%，交互项 +2.13%。说明两者互补：改写把网表整理成 mux 树后，ABC 的结构优化能继续收敛；此后追加 &syn3 又把总平均推到 20.41%。

第四部分 总结

4.1 可扩展之处

一是把“常量解码”进一步扩展到被 pmuxtree 拆成 $mux 链的一热结构，在更晚阶段再抓一批结构；二是把门限从“覆盖率/链深”推广为带代价模型的收益预估，用比较器数与树规模直接估算 LUT 差；三是探索 post-ABC 的等价重构（赛题明确不允许，本作品

未做，仅作为后续研究方向）；四是将同一套改写抽象为工艺无关的 pass，接入其它厂商的映射流程。

4.2 心得体会

这次工作中最有价值的经验，是“数值上的收益必须能被形式化证明”。项目早期曾出现一版让 test1 从 288 降到 261 的“好结果”，但等价性检查立刻指出 28 个单元无法证明——原因是对 $eq 位宽扩展理解错误，把窄常量当成了全 1 常量。如果只看面积数字，这个错误会一路带进最终提交；正是“先证明、再报数”的顺序救回了它。第二个体会是“小心 RTLIL 的语义细节”。$mux 的 A/B 极性、$pmux 的下标优先、以及 &save 之前必须有 &if，这些都不是文档里一眼能看出的东西，只能靠最小用例逐向量比对、逐条命令试算来确认。第三个体会是“优化要能被约束解释”。我们最终没有采用“按用例切脚本”“双跑两套脚本取最小”这类看起来有效的做法，因为它们要么违反统一脚本的要求，要么突破 5% 的时间线；最后留下的是两条纯结构性门限和一条经过测量的统一脚本，任何一条都能在报告里写清楚为什么。第四个体会是工程纪律：每次改动都跑 9 个用例的面积、时间、内存与等价性回归，并把失败的方向、数据与原因都记进项目文档，避免在同一个坑里反复试探。回头看，真正让优化率从 0 走到 20.41% 的，不是某一个巧妙的技巧，而是“结构改写 + 映射调优 + 严格验证”三件事的叠加与反复校准。

第五部分 参考文献

[1] Yosys Open SYnthesis Suite 文档：synth_intel 与 abc9 pass 说明.

[2] Berkeley Logic Synthesis and Verification Group. ABC: A System for Sequential Synthesis and Verification.

[3] Wolf C. Yosys Manual（RTLIL 中间表示与 pass 编写规范）.

[4] Mishchenko A., et al. Improvements to Combinational Equivalence Checking. ICCAD, 2006.

[5] Brayton R., Mishchenko A. ABC: An Academic Industrial-Strength Verification Tool. CAV, 2010.

[6] 赛事组委会. FPGA 创新大赛问题汇总（选题二答疑，含 abc 脚本与计时口径）.